KNOWLEDGE
INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)

SALEM – 637 504

DEPARTMENT

OF

ELECTRONICS AND COMMUNICATION ENGINNERING

LABORATORY RECORD

Practical Code	**::**	BE23EE512
Practical Name	**::**	ARM Processors
Year / Sem	**::**	III/V
Academic Year	**::**	2026 – 2027 (Odd Sem)
Regulation	**::**	R2023

Name of the Student	**::**	
Register Number	**::**	

KNOWLEDGE INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NAAC "A" Grade & NBA (EEE, ECE, CSE & MECH),
Approved by AICTE New Delhi, Affiliated to Anna University Chennai
NH – 47, KIOT Campus, Kakapalayam (PO), Salem – 637 504

RECORD NOTE BOOK

REGISTER NUMBER

Certified that this is the bonafide record of work done by Selvan / Selvi ……..……of the ***V*** *Semester* ***B.E.–Electronics and Communication Engineering*** *Branch during* ***the year 2026 – 2027 in the BE23EE512- ARM processors Laboratory.***

Faculty In charge **Head of the Department**

Submitted for the End Semester Practical Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Electronics and Communication Engineering	
Vision	
✓ To produce competent Electronics and Communication Engineers by imparting quality education to meet the industry requirements and for serving the societal needs	
Mission	
✓ To develop appropriate facilities for promoting research activities ✓ To inculcate leadership qualities among students for self and societal growth ✓ To nurture students on emerging technologies for serving industry needs through industry institute interface ✓ To enrich teaching learning process by transforming young minds to be resourceful engineers	
Programme Educational Objectives (PEOs) The graduates of Electronics and Communication Engineering will be able to	
PEO – 1	To enable graduates to pursue research, or have a successful career in academia or industries associated with Electronics and Communication Engineering, or as entrepreneurs
PEO – 2	To provide students with strong foundational concepts and also advanced techniques and tools in order to enable them to build solutions or systems of varying complexity
PEO – 3	To prepare students to critically analyze existing literature in an area of specialization and ethically develop innovative and research oriented methodologies to solve the problems identified
Programme Specific Outcome (PSOs) The graduates of Electronics and Communication Engineering will be able to	
PSO – 1	Use signal processing concepts and tools to provide solutions to real time problems
PSO – 2	Use embedded system concepts for developing IoT applications
PSO – 3	Use the concepts of analog and digital electronics to design and implement VLSI circuits

Programme Outcomes (POs) The graduates of Electrical and Electronics Engineering will be able to:	
PO – 1	**Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO – 2	**Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO – 3	**Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO – 4	**Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO – 5:	**Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO – 6:	**The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO – 7:	**Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO – 8:	**Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO – 9:	**Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.
PO – 10:	**Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO – 11:	**Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO – 12:	**Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.
Course Outcomes: At the end of the course, the student should be able to	
CO1	Explain the architecture, modes, and instruction sets of ARM processors.
CO2	Write and debug ARM assembly programs using simulation and hardware.
CO3	Interface peripherals with ARM and build real-time embedded applications.
CO4	Explain ARM SoC design considerations, memory architecture, and interrupt mechanisms.
CO5	Implement and evaluate ARM features like NEON, TrustZone, and low-power techniques in practical scenarios.

Rubrics for Mark Allocation

Criteria	Max Marks (100)	Excellent (Full Marks)	Good (75%)	Satisfactory (50%)	Needs Improvement (25%)
Preparation & Pre-Lab Work	20	Thorough understanding of experiment objectives, theory, ARM7 (LPC2129) architecture, registers, peripherals (GPIO, Timer, ADC, DAC, UART), ARM assembly instructions and interfacing devices (LED, push button, potentiometer, 16x2 LCD). Familiar with tools (Keil µVision, Flash Magic). Complete pre-lab notes prepared.	Minor gaps in theory or procedure; pre-lab work mostly prepared.	Basic understanding of experiment; incomplete pre-lab notes.	Unprepared, no notes.
Algorithm & Flowchart / Circuit Diagram	15	Clear and correct algorithm/flowchart with accurate LPC2129 interfacing circuit diagram (e.g., LED with resistor, push button, potentiometer to ADC pin, DAC output, 16x2 LCD data/control lines). Proper pin configuration (PINSEL, IODIR, IOSET/IOCLR mapping), symbols, and labeling used.	Minor errors in logic or pin connections; mostly clear.	Basic algorithm or circuit drawn but incomplete or unclear.	Incorrect or missing algorithm/flowchart/circuit diagram.
Program Code	10	Correct and efficient Embedded C / ARM assembly program for LPC2129, using proper registers (PINSEL, IODIR, IOSET, IOCLR, T0TC, ADCR, DACR, U0THR, etc.), delay routines, branching, subroutines, and LCD commands. Well-structured, modular, and properly commented.	Minor syntax or logical errors; program understandable.	Program runs with frequent guidance; limited explanation of code.	Incorrect program; unable to explain logic or instructions.
Program Execution & Output	25	Successful build in Keil with zero errors and real-time execution on LPC2129 with correct output (e.g., LED blinking at required rate, 1–100 s variable delay, ADC value on serial monitor with DAC output, R0 holding correct 32-bit sum with push button control, LCD string display, two LEDs alternating at 500 ms).	Output obtained with minor issues or corrections.	Partial output obtained with frequent instructor assistance.	Program fails to execute or output is incorrect.
Results & Interpretation	10	Accurate results with proper technical interpretation (e.g., delay calculation from PCLK and timer count, ADC resolution and voltage conversion, DAC output, register contents and flags after addition, LCD command timing, 500 ms blink verification).	Results correct with brief explanation.	Results noted with weak interpretation.	Incorrect or missing results and justification.
Teamwork & Laboratory Etiquette	10	Works collaboratively; proper handling of hardware kits, power supply, LEDs, LCD, potentiometer and push button; follows lab safety and discipline strictly.	Works well with minor issues.	Minimal participation; occasional carelessness in handling equipment.	Disruptive behavior; unsafe or improper lab practices.
Oral Viva / Understanding	10	Deep conceptual understanding of ARM7 architecture, LPC2129 registers, GPIO, timers, ADC/DAC, UART and LCD interfacing; answers confidently with application-level knowledge.	Most answers correct with good understanding.	Basic answers; lacks depth and clarity.	Unable to answer; poor conceptual understanding.

CONTENTS

S.No.	Date	Name of the Experiments	Page No	Marks Awarded	Faculty Sign
1		Introduction to Keil IDE and Blink Programming in ARM			
2		Delay Loop Programming			
3		DAC Programming			
4		Assembly Language Program for Arithmetic Operation			
5		LCD Interfacing Program			
6		ARM Programming			
7					
			AVG		

PROGRAM:

```
#include <lpc214x.h>
#define LED1 8
/* Delay for seconds */
void delay_s(unsigned int dlyS)
{
    dlyS *= 12000000;         // Each second = 1,000 ms
    while(dlyS--);     // Decrement the dlyS value until it reaches 0.
}
int main()
{
   IODIR0 |= (1 << LED1);/* Configure P0.8 as output pin */
   while(1)
   {
      IOCLR0 = (1 << LED1);// Active-low LED ON
      delay_s(1);//delay
      IOSET0 = (1 << LED1);// Active-low LED OFF
      delay_s(1);//delay
   }
}
```

Ex. No: 01 Date:	Introduction to Keil IDE and Blink Programming in ARM

AIM:

To write program in ARM Microcontroller using Keil IDE to blink a LED.

HARDWARE & SOFTWARE REQUIRED:

S.No.	Hardware Requirements	Software Requirements	Quantity
1.	LPC2129 ARM7TDMI-S Microcontroller	Keil µVision4 IDE	1
2.	Power Supply	Flash Magic	1
3.	Connecting cables	Proteus Simulation Software	1

PROCEDURE:

Step 1: Open Keil µVision IDE and create a new project for LPC2129.

Step 2: Write the program to blink the LED at 1000 ms delay.

Step 3: Configure IODIR0 register to set LED pin (P0.8) as output.

Step 4: Use IOCLR0 to turn LED ON (active-low) and IOSET0 to turn LED OFF.

Step 5: Introduce a software delay of 1000 ms using nested loops.

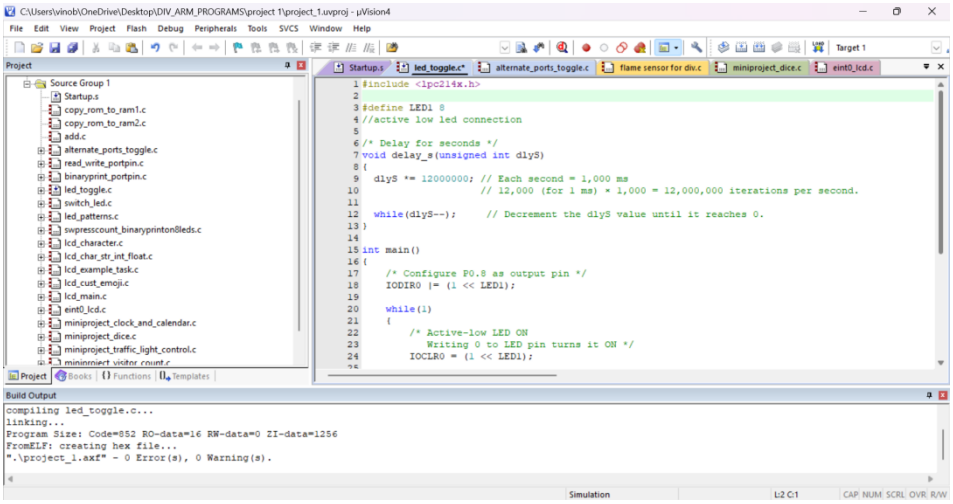
Step 6: Compile the program and check for errors/warnings.

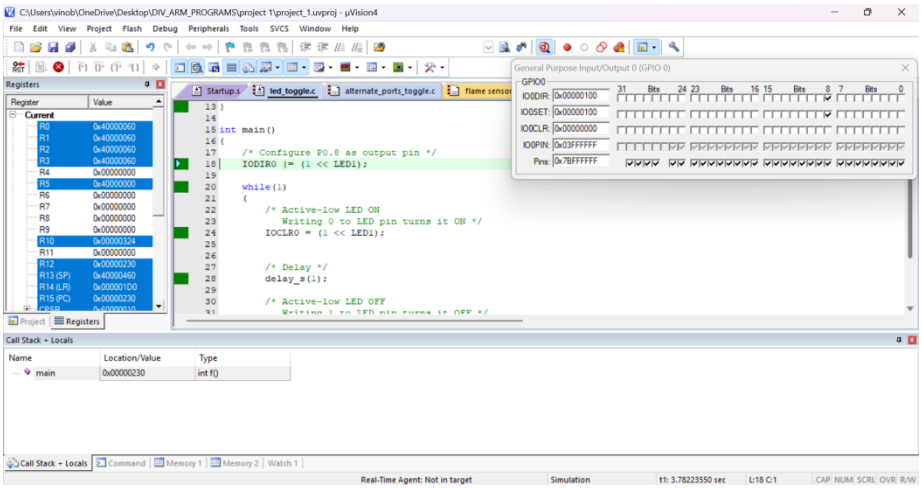
Step 7: Debug the program in Keil (simulate step by step if required).

Step 8: If using actual hardware, load the HEX file into LPC2129 using Flash Magic observe LED blinking at 1000 ms interval.

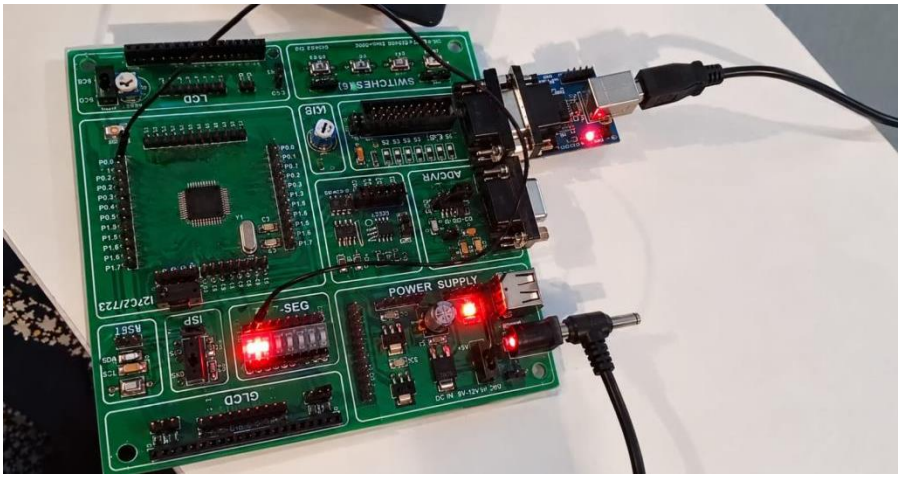
OUTPUT:

KEIL OUTPUT:





LPC2129 OUTPUT:



INFERENCE:

RESULT:

Assessment Parameters	Max. Marks	Marks Obtained
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

PROGRAM:

```
#include <LPC21xx.h>
#define TIMER0_ENABLE 0x01  //Bit 0 = 1 ->(TC) and (PC)
#define TIMER0_RST    0x02  //Bit 1 = 1 -> Reset TC & PC to 0
#define TIMER0_INT_ON_MR0  0x01  //Interrupt on MR0 match
#define TIMER0_RST_ON_MR0  0x02  //Reset TC on MR0 match
#define TIMER0_STOP_ON_MR0 0x04  //Stop TC on MR0 match
void InitTimer0(void)
{
   T0TCR = TIMER0_RST;        // Reset Timer0 counters (TC & PC) to 0
   T0MCR = TIMER0_STOP_ON_MR0;// Stop Timer0 when MR0 matches TC
}
void tdelay_us(unsigned int microseconds)
{
   T0MR0 = microseconds;      // Match after 'microseconds' ticks
   T0PR  = 30 - 1;       // Prescaler: 30MHz PCLK -> 1 tick = 1  s
   T0TC  = 0;            // Reset Timer Counter
   T0TCR = TIMER0_ENABLE;// Start Timer0
   while (T0TC != T0MR0);// Wait until TC reaches match value
}
void tdelay_ms(unsigned int milliseconds)
{
   T0MR0 = milliseconds;   //Match value in milliseconds
   T0PR  = 30000 - 1;      //Prescaler: 30MHz/1000Hz -> 1 tick = 1 ms
   T0TC  = 0;              //Reset Timer Counter
   T0TCR = TIMER0_ENABLE;  //Enable Timer0
   while (T0TC != T0MR0);  //Wait until TC reaches match value
}
void tdelay_s(unsigned int seconds)
{
   T0MR0 = seconds;         //Match value in seconds
   T0PR  = 30000000 - 1;    //Prescaler: 30MHz/1Hz -> 1 tick = 1 second
   T0TC  = 0;               //Reset Timer Counter
   T0TCR = TIMER0_ENABLE;   //Enable Timer0
   while (T0TC != T0MR0);   //Wait until TC reaches match value
```

Ex. No: 02 Date:	Delay Loop Programming

AIM:

To write a Program in ARM microcontroller to generate delays in microseconds, milliseconds, seconds.

HARDWARE & SOFTWARE REQUIRED:

S.No.	Hardware Requirements	Software Requirements	Quantity
1.	LPC2129 ARM7TDMI-S Microcontroller	Keil µVision4 IDE	1
2.	Power Supply	Flash Magic	1
3.	Connecting cables	Proteus Simulation Software	1

PROCEDURE:

Step 1: Open Keil µVision IDE and create a new project for LPC2129.

Step 2: Write the program to generate delays in microseconds, milliseconds, seconds using Timer0.

Step 3: Initialize Timer0 by setting the prescale Register (T0PR = 14999) to obtain a 1 ms time base from a 15 MHz peripheral clock.

Step 4: Reset and enable Timer0 using the Timer Control Register (T0TCR).

Step 5: Write a delay function tdelay_us(), tdelay_ms(), tdelay_s that calls Timer0 in a loop to generate delays in microseconds, milliseconds, seconds.

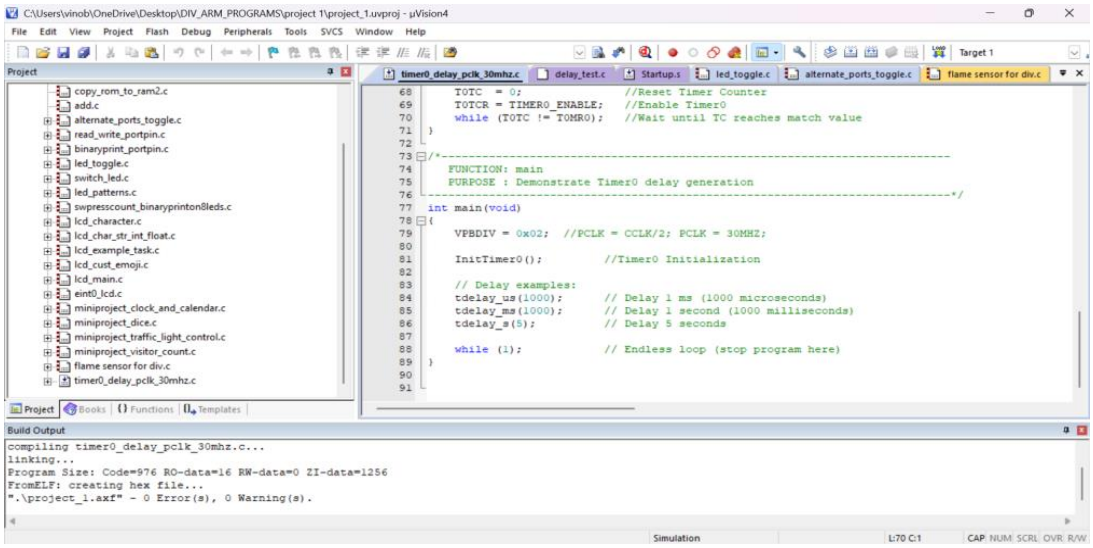
Step 6: Compile the program and check for errors or warnings.

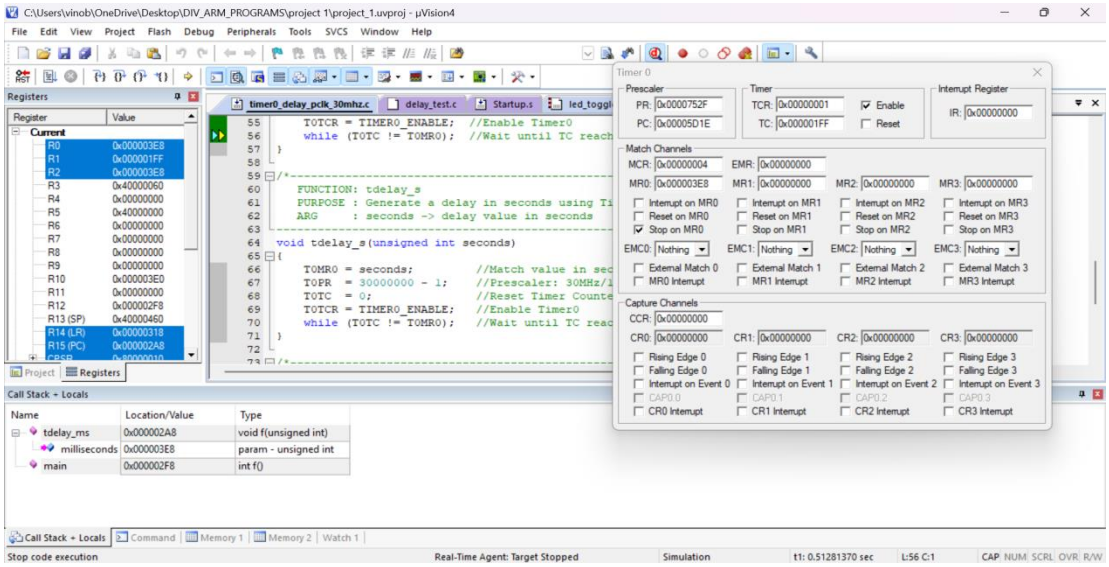
Step 7: Load the generated HEX file into LPC2129 using Flash Magic and observe the LED blinking with variable ON and OFF times (1–100 seconds).

```
}
int main(void)
{
  VPBDIV = 0x02;        //PCLK = CCLK/2; PCLK = 30MHZ;
   InitTimer0();        //Timer0 Initialization
   tdelay_us(1000);     // Delay 1 ms (1000 microseconds)
   tdelay_ms(1000);     // Delay 1 second (1000 milliseconds)
   tdelay_s(5);         // Delay 5 seconds
   while (1);           // Endless loop (stop program here)
}
```

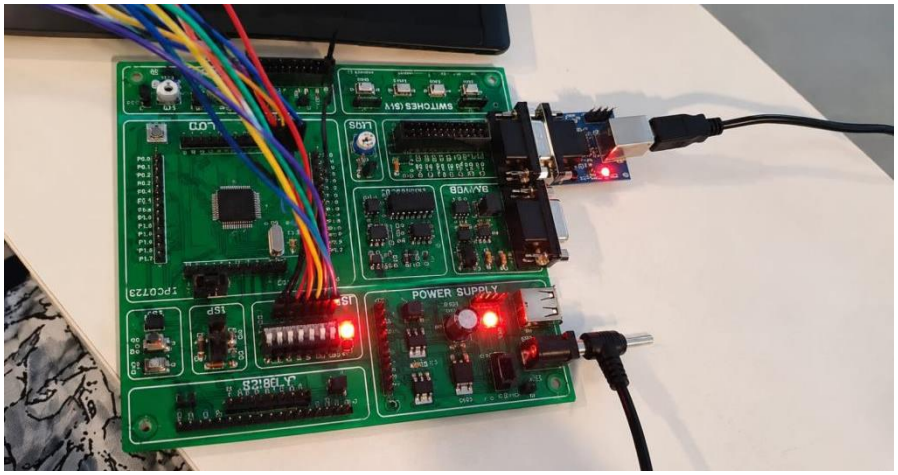
OUTPUT:

KEIL OUTPUT:





LPC2129 OUTPUT:



INFERENCE:

RESULT:

Assessment Parameters	Max. Marks	Marks Obtained
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

PROGRAM:

```
#include <LPC214x.h>
// Function to initialize DAC
void DAC_Init(void) {
    PINSEL1 |= 0x00080000;  // Select P0.25 as AOUT (DAC output)
                                    // Set bits 19:18 = 10
}
// Function to write value to DAC
void DAC_Write(unsigned int value) {
    unsigned int dac_value;

    // DAC value must be 10-bit (0-1023)
    // Left shift by 6 bits as DAC register format is [15:6]
    dac_value = (value & 0x3FF) << 6;

    DACR = dac_value;  // Write to DAC register
}
// Delay function
void delay_ms(unsigned int ms) {
    unsigned int i, j;
    for(i = 0; i < ms; i++)
        for(j = 0; j < 6000; j++);
}
int main(void) {
    unsigned int dac_output = 0;

    DAC_Init();  // Initialize DAC

    while(1) {
        // Generate sawtooth wave
        for(dac_output = 0; dac_output < 1024; dac_output++) {
            DAC_Write(dac_output);
            delay_ms(1);  // Small delay for visible output
        }
    }
```

Ex. No: 03 Date:	DAC Programming

AIM:

To interface a potentiometer with the ADC of the microcontroller, read its analog value, display the reading on a serial monitor, and generate a corresponding analog output through the DAC to control an external analog device.

HARDWARE & SOFTWARE REQUIRED:

S.No	Hardware Requirements	Software Requirements	Quantity
1.	LPC2129 ARM7TDMI-S Microcontroller	Keil µVision4 IDE	1
2.	Potentiometer (variable resistor)	Flash Magic(For programming)	1
3.	DAC / External Analog Device	Proteus Simulation Software	1
4.	Power Supply (5v as per board)	-	1

PROCEDURE:

Step 1: Open Keil µVision and create a new project for LPC2129.

Step 2: Configure ADC registers to read potentiometer input.

Step 3: Initialize UART to display ADC values on the serial monitor.

Step 4: Read ADC values periodically in the program.

Step 5: Convert ADC readings to a suitable range for DAC output.

Step 6: Configure DAC registers to output corresponding analog voltage.

Step 7: Introduce a small software delay if needed.

Step 8: Compile the program and check for errors/warnings.

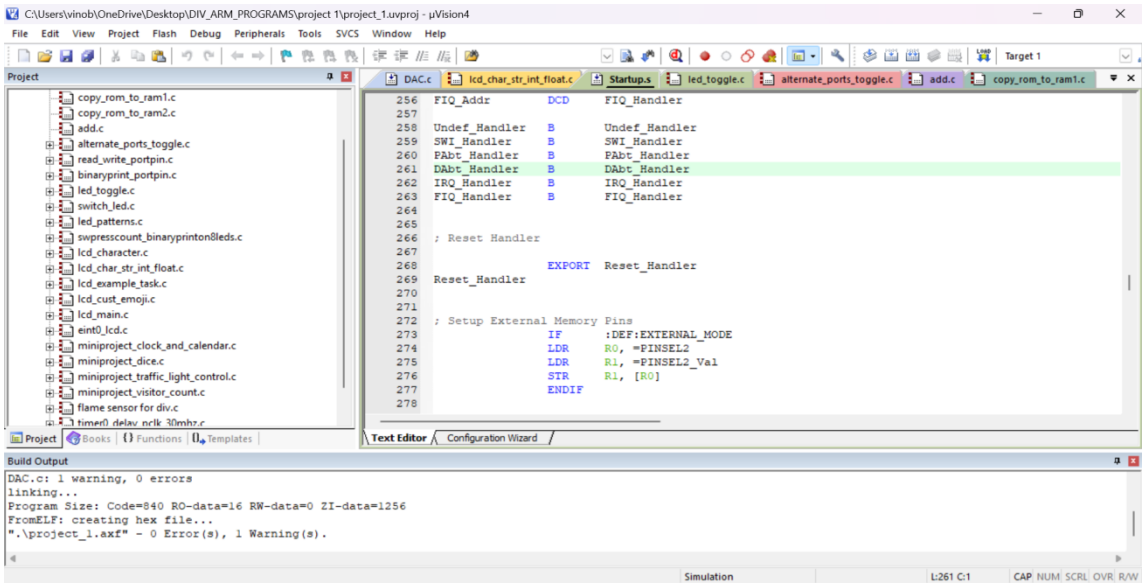
Step 9: Debug the program in Keil using simulation.

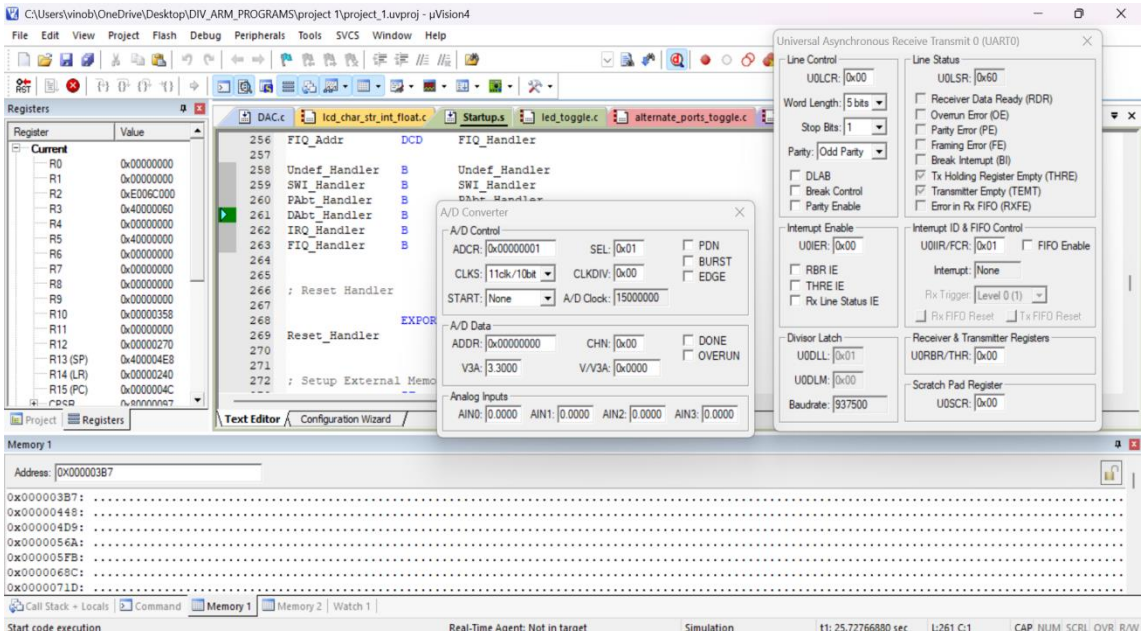
Step 10: Load HEX to LPC2129 via Flash Magic and observe ADC/DAC outputs.

```
    return 0;
}
```

OUTPUT:

KEIL OUTPUT:





INFERENCE:

RESULT:

Assessment Parameters	Max. Marks	Marks Obtained
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

PROGRAM:

```
AREA MYSEG,CODE,READONLY
ENTRY
MOV R1,#0X10
MOV R2,#0X20
ADD R0,R1,R2
END
```

Ex. No: 04 Date:	Assembly Language Program for Arithmetic Operation

AIM:

To write an assembly program that adds two 32-bit integers, stores the result in R0, implements branching and subroutines, and interfaces a push button to control outputs.

HARDWARE & SOFTWARE REQUIRED:

S.No	Hardware Requirements	Software Requirements	Quantity
1.	LPC2129 ARM7TDMI-S Microcontroller	Keil µVision4 IDE	1
2.	Push Button / Switch	Proteus Simulation Software	1
3.	LEDs (for output observation)		1
4.	Power Supply (5V)	Flash Magic	

PROCEDURE:

Step 1: Open Keil µVision and create a new project for LPC2129.

Step 2: Configure the GPIO pin connected to the push button as input.

Step 3: Configure the GPIO pin(s) connected to LEDs as output.

Step 4: Write assembly code to read the push button state.

Step 5: Implement addition of two 32-bit integers and store result in R0.

Step 6: Implement branching and loops as needed in the program.

Step 7: Include subroutine calls to organize repeated operations.

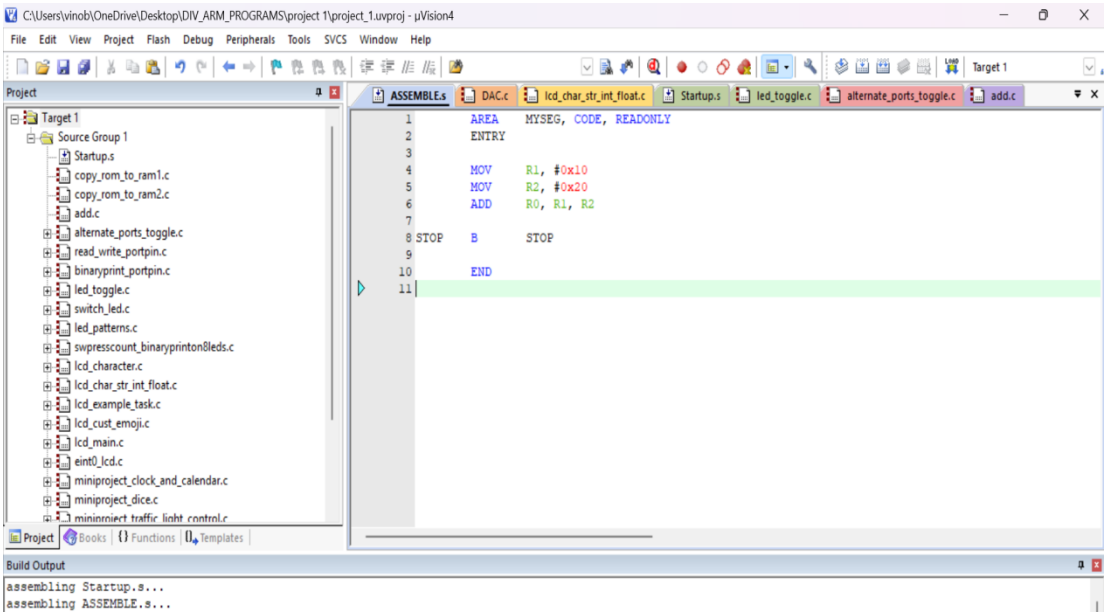
Step 8: Compile the program and check for errors or warnings.

Step 9: Debug the program in Keil using the simulator.

Step 10: Load HEX to LPC2129 via Flash Magic and test push button and LED outputs.

OUTPUT:

KEIL OUTPUT:



INFERENCE:

RESULT:

Assessment Parameters	Max. Marks	Marks Obtained
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

PROGRAM:

```
#include <LPC21xx.h>
#define LCD_DATA   8        // LCD D0 starts from P0.8
#define LCD_RS     16       // LCD RS connected to P0.16
#define LCD_RW     17       // LCD RW connected to P0.17
#define LCD_EN     18       // LCD EN connected to P0.18
void delay_us(unsigned int);
void delay_ms(unsigned int);
void WriteLCD(unsigned char);
void CmdLCD(unsigned char);
void InitLCD(void);
void StrLCD(char *);
int main()
{
   InitLCD();
   CmdLCD(0x01);//Display string.
   StrLCD("EMBEDDED");
   delay_ms(1000);
  while(1);
}
void delay_us(unsigned int dlyUS)
{
dlyUS *= 12;
while(dlyUS--);  // Decrement then dlyUS value until it reaches 0.
}
/* Delay for milliseconds */
void delay_ms(unsigned int dlyMS)
{
dlyMS *= 12000;
while(dlyMS--); // Decrement the dlyMS value until it reaches 0.
}
void WriteLCD(unsigned char byte)
{
IOCLR0 = 1 << LCD_RW;
IOPIN0 = (IOPIN0 & ~(0xFF << LCD_DATA)) | (byte << LCD_DATA);
```

Ex. No: 05 Date:	LCD Interfacing Program

AIM:

To write an Embedded C Program (ECP) to display a string on a 16x2 alphanumeric LCD using LPC2129.

HARDWARE & SOFTWARE REQUIRED:

S.No	Hardware Requirements	Software Requirements	Quantity
1.	LPC2129 Development Board	Keil µVision 4.0	1
2.	16x2 LCD	Flash magic	1
3.	Connecting wires	Proteus simulation	As required

PROCEDURE:

Step 1: Connect the 16x2 LCD to LPC2129 (data → P0.8–P0.15, RS=P0.16, EN=P0.18).

Step 2 : Configure the LCD pins as output using IODIR0.

Step 3 : Initialize the LCD with proper commands (clear display, cursor, 8-bit mode).

Step 4 : Send commands to set cursor position.

Step 5 : Send characters to LCD using data function.

Step 6 : Display the string "EMBEDDED".

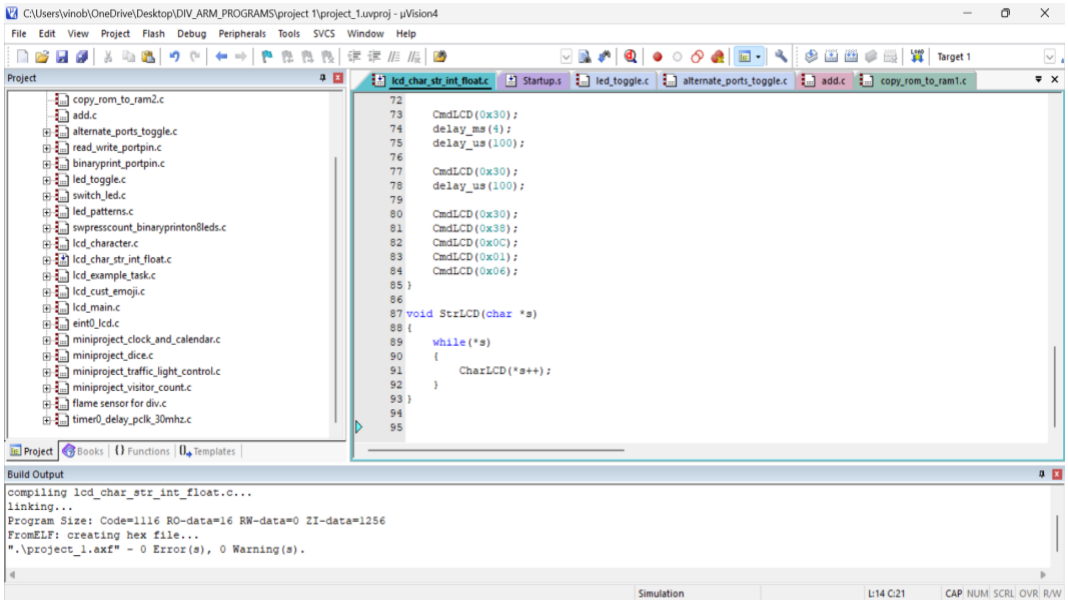
Step 7 : Run the program using Keil uVision.

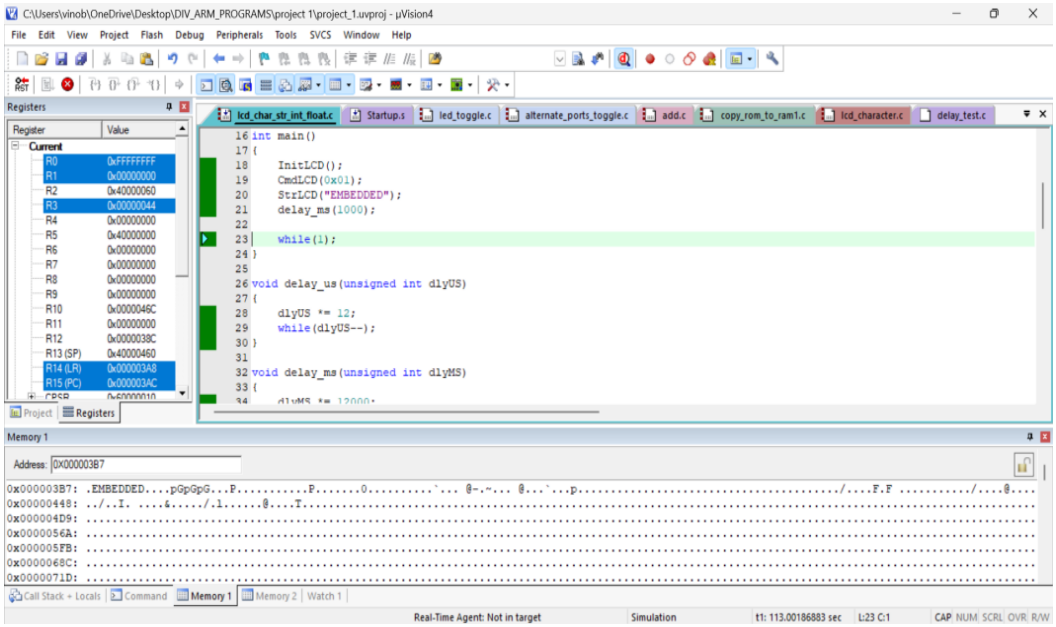
Step 8 : Download the hex file to LPC2129 using Flash Magic.

```
 IOSET0 = 1 << LCD_EN;
 delay_us(1);
 IOCLR0 = 1 << LCD_EN;
 delay_ms(2);
 }
void CmdLCD(unsigned char cmd)
{
   IOCLR0 = 1 << LCD_RS;
   WriteLCD(cmd);
}
void InitLCD(void)
{
  IODIR0 |= (0xFF << LCD_DATA) | (1 << LCD_RS) | (1 << LCD_RW)
          | (1 << LCD_EN);
   delay_ms(15);
   CmdLCD(0x30);
   delay_ms(4);
   delay_us(100);
   CmdLCD(0x30);
   delay_us(100);
   CmdLCD(0x30);
   CmdLCD(0x38);
   CmdLCD(0x0C);
   CmdLCD(0x01);
   CmdLCD(0x06);
}
void StrLCD(char *s)
{
   while(*s)
   {
      CharLCD(*s++);
  }
}
```

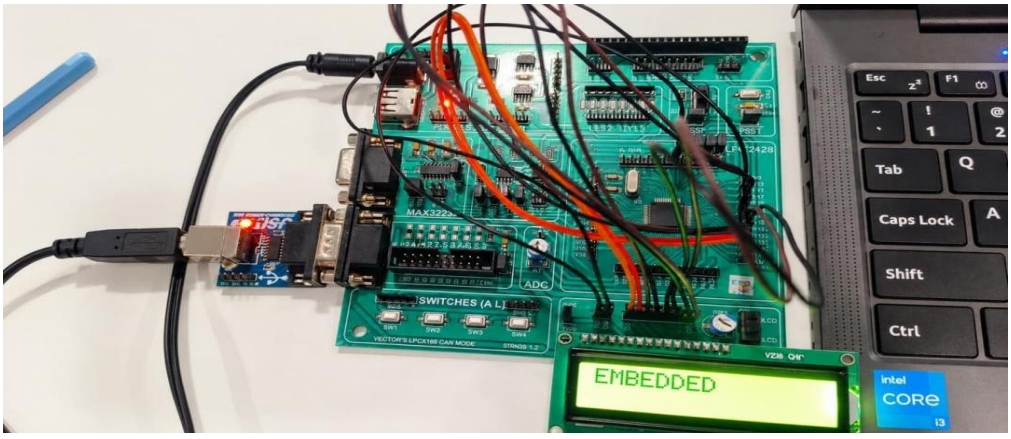
OUTPUT:

KEIL OUTPUT:





LPC2129 OUTPUT:



INFERENCE:

RESULT:

Assessment Parameters	Max. Marks	Marks Obtained
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

PROGRAM:

```
#include <lpc214x.h>
#define LED1 8
#define LED2 9
#define LED3 10//active low led connection
/* Delay for seconds */
void delay_s(unsigned int dlyS)
{
      dlyS *= 6000000;
      while(dlyS--);
}
int main()
{
   /* Configure P0.8 and P0.9 as output pins */
   IODIR0 |= (1 << LED1) | (1 << LED2);
   while(1)
   {
      /* LED1 ON, LED2 OFF */
      IOCLR0 = (1 << LED1);
      IOSET0 = (1 << LED2);
      delay_s(1);
      /* LED1 OFF, LED2 ON */
      IOSET0 = (1 << LED1);
      IOCLR0 = (1 << LED2);
      delay_s(1);
   }
}
```

Ex. No: 06 Date:	ARM Programming

AIM:

To write a Program to blink 2 LEDs alternately at 500ms delay using LPC2129.

HARDWARE & SOFTWARE REQUIRED:

S.No	Hardware Requirements	Software Requirements	Quantity
1.	LPC2129 Development Board	Keil u vision 4 Flash Magic	1
2.	LED	-	2
3.	Connecting wires	-	As required

PROCEDURE:

Step 1: Connect two LEDs to LPC2129 at P0.8 and P0.9 through current limiting resistors.

Step 2: Configure P0.8 and P0.9 as output using IODIR0.

Step 3: Use IOSET0 and IOCLR0 to turn ON/OFF LEDs.

Step 4: Implement a delay function using Timer0.

Step 5: Turn ON LED1 and turn OFF LED2.

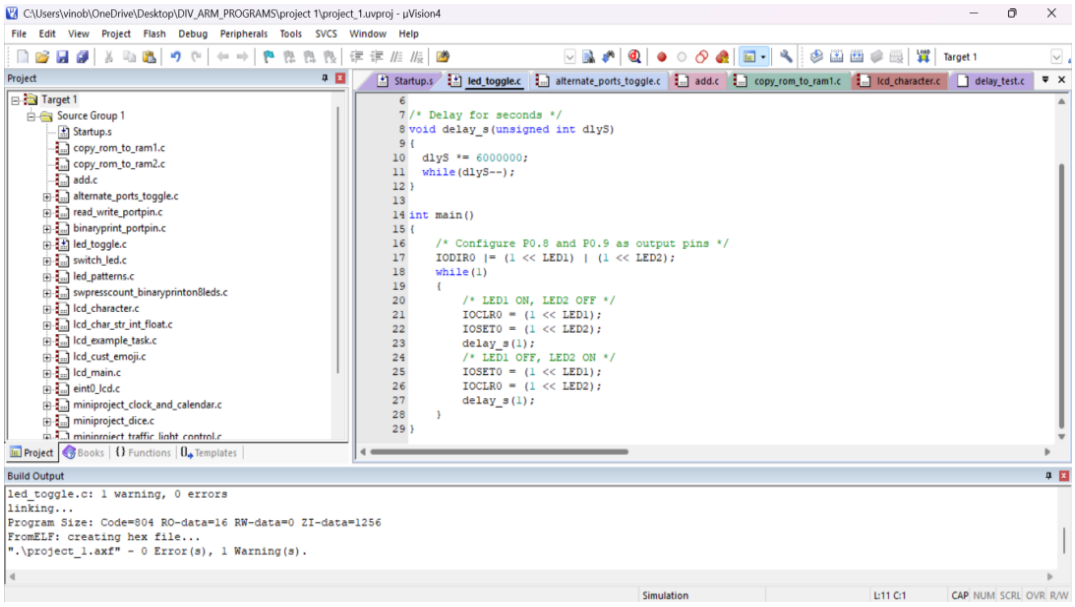
Step 6: Wait for 500ms.

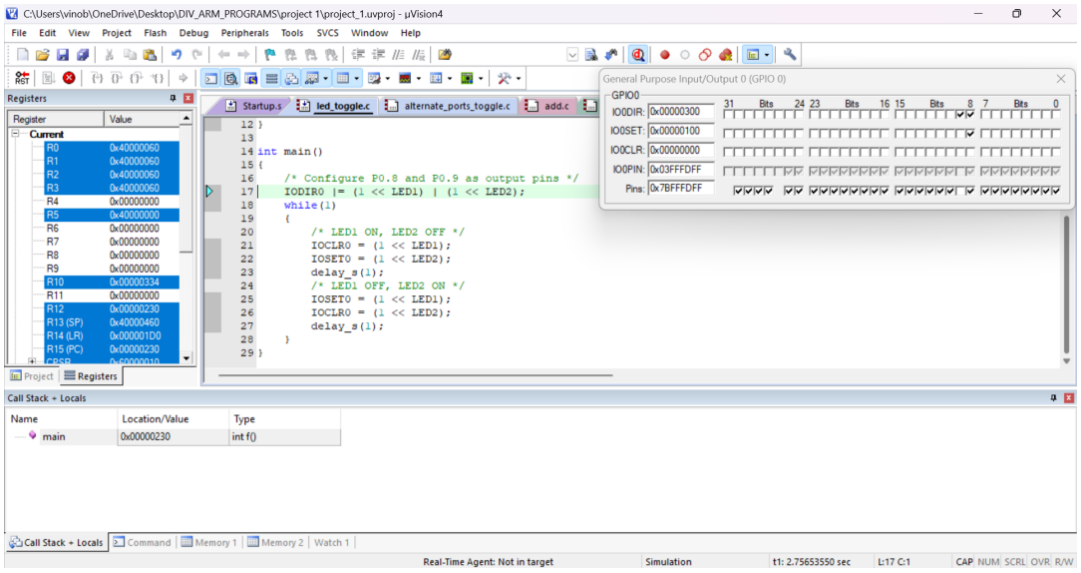
Step 7: Turn ON LED2 and turn OFF LED1.

Step 8: Repeat steps 5–7 infinitely.

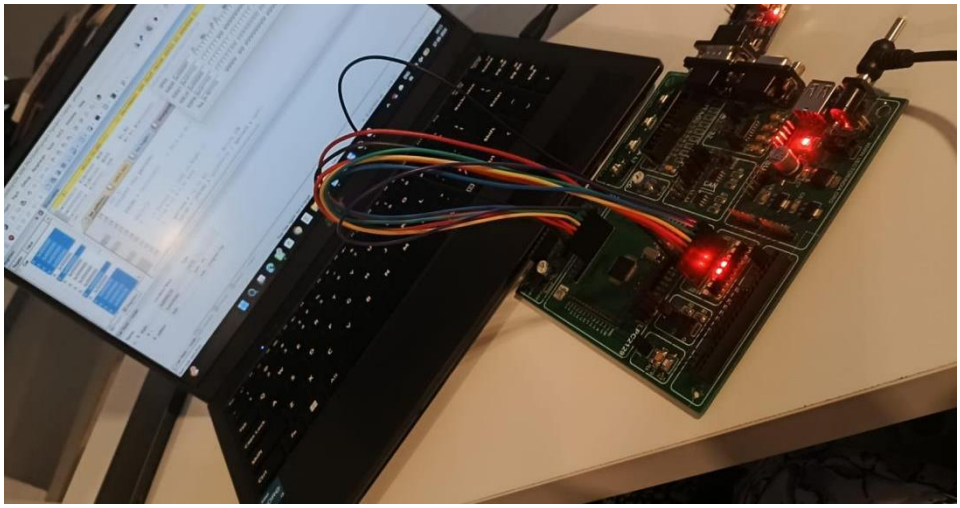
OUTPUT:

KEIL OUTPUT:





LPC2129 OUTPUT:



INFERENCE:

RESULT:

Assessment Parameters	Max. Marks	Marks Obtained
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		

Ex. No: 07 Date:	Mini Project

Title:

Assessment Parameters	Max. Marks	Marks Obtained
Preparation & Pre-lab Work	10	
Algorithm & Flowchart / Circuit Diagram	15	
Program Code	20	
Program Execution & Output	25	
Results & Interpretation	10	
Teamwork & Laboratory Etiquette	10	
Oral Viva / Understanding	10	
Total	**100**	
Faculty Signature with Date		